UNIVERSITY OF GHANA

COLLEGE OF BASIC AND APPLIED SCIENCES



CAMPUS COMPANION: AN INTEGRATED MOBILE APPLICATION FOR ACADEMIC SCHEDULING, GRADE ANALYTICS AND CAMPUS-LIFE MANAGEMENT FOR GHANAIAN TERTIARY STUDENTS

BY

JUVELYN EKOW SACKEY

(11263517)

A PROJECT WORK SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE, UNIVERSITY OF GHANA, LEGON, IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF BACHELOR OF SCIENCE (BSc) DEGREE IN INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

OCTOBER, 2026

DECLARATION

I hereby declare that this project, “Campus Companion: An Integrated Mobile Application for Academic Scheduling, Grade Analytics and Campus-Life Management for Ghanaian Tertiary Students”, is the result of my own work, carried out under the supervision of Mr. Solomon Mensah in the Department of Computer Science, University of Ghana.

Except where other people's work has been duly acknowledged by reference, this work is my own, and it has not been submitted, in whole or in part, for the award of any other degree, diploma or qualification in this or any other institution.

STUDENT

Name: Juvelyn Ekow Sackey

Signature: .....

Date: .....

SUPERVISOR

Name: Mr. Solomon Mensah

Signature: .....

Date: .....

ABSTRACT

Context

Ghanaian university students manage timetables, grades, campus announcements, events and revision material using a patchwork of paper diaries, generic calendar apps, and social-media groups, none of which is built around the specific academic conventions of their institutions — such as the University of Ghana's letter-grade and GPA scale, under which a mark of 45–49 does not count as a pass. No evaluated application in the literature surveyed combines personal academic organisation and campus-wide services behind one institution-aware, authenticated account.

Aim

This study aimed to design, implement and evaluate Campus Companion, an integrated mobile-first application that consolidates academic scheduling, grade analytics and campus-life services for Ghanaian tertiary students within a single secure platform reflecting their institutions' actual academic conventions.

Method

Requirements were identified through the developer's own analysis of university students' academic and campus-life needs; no interviews or surveys were conducted. Campus Companion was built as a Flutter mobile client backed by a Node.js/Express REST API on PostgreSQL, developed iteratively and feature by feature, with each feature's backend implemented and tested before its mobile client code. Correctness was verified through three automated test tiers — offline unit tests, database-backed integration tests, and end-to-end smoke tests — reinforced by deliberate mutation testing of key business rules, and cross-checked against a live Firebase credential for Google Sign-In.

Result

Re-running every test tier on 30 October 2026 produced 371/371 mobile tests, 50/50 backend unit tests, 14/14 database tests and 182/182 end-to-end smoke-test checks; running the application end to end also exposed three user-interface defects that the automated tests had missed, all of which were fixed and are now covered by regression tests. The specific objectives were substantially met at the working-prototype level (Table 4.2), but two — the AI assistant's behaviour against a live large-language-model key and on a device, and production delivery of push notifications and Google Sign-In — are code-complete and tested against fakes but not

yet observable end-to-end, pending configuration steps outside the application's own code. No formal usability evaluation with students has yet been conducted.

Conclusion

The fragmentation of academic and campus-life tools available to Ghanaian tertiary students can be addressed with a single institution-aware application whose correctness is demonstrated through repeatable automated testing rather than asserted; the remaining gaps identified in this study are deployment and configuration work, not unresolved design problems.

Keywords

mobile application; academic scheduling; grade point average; AI assistant; Ghana; Flutter

DEDICATION

I dedicate this project to God Almighty for His grace, strength and guidance throughout my academic journey. I also dedicate this work to my parents and family for their unconditional love, sacrifices, prayers, encouragement and unwavering support throughout my education.

ACKNOWLEDGEMENT

First and foremost, I express my sincere gratitude to God Almighty for His grace, strength, wisdom and guidance throughout my academic journey and during the development and completion of this project.

I extend my heartfelt appreciation to my supervisor, **Mr. Solomon Mensah**, for his guidance, constructive criticism, patience and continuous support throughout this project. His knowledge, advice and encouragement provided valuable direction during the planning, design, development and documentation of Campus Companion.

My sincere thanks also go to all lecturers in the Department of Computer Science, University of Ghana, for imparting the knowledge, skills and professional values that have contributed to my academic and personal development.

I am deeply grateful to my parents and family for their unconditional love, prayers, sacrifices, encouragement and financial support throughout my education. Their belief in me has remained a constant source of motivation and strength.

I also wish to acknowledge my colleagues, friends and classmates for their encouragement, helpful suggestions and support throughout the development of this project.

Finally, I sincerely appreciate everyone who contributed in diverse ways, directly or indirectly, towards the successful completion of this project. May God richly bless you all.

TABLE OF CONTENTS

DECLARATION i
ABSTRACT ii
DEDICATION iv
ACKNOWLEDGEMENT v
TABLE OF CONTENTS vi
LIST OF FIGURES ix
LIST OF TABLES x
LIST OF ABBREVIATIONS xi

CHAPTER ONE: INTRODUCTION 1
1.1 Background and Motivation 1
1.2 Statement of Problem 1
1.3 Scope of the Study 2
1.4 Research Objectives 3
1.4.1 Global Objective 3
1.4.2 Specific Objectives 3
1.5 Research Contribution 4
1.6 Organization of the Study 4

CHAPTER TWO: LITERATURE REVIEW 6
2.1 Introduction 6
2.2 Mobile Applications for Academic Scheduling and Student Life Management 6
2.3 Mobile Learning and Smartphone Use Among Ghanaian University Students 7
2.4 Authentication and Security in Mobile Applications 8
2.5 Push Notification Architectures in Mobile Systems 9
2.6 AI-Assisted Virtual Assistants and Chatbots in Education 9
2.7 Review of Existing Systems 10
2.8 Summary and Research Gap 11

CHAPTER THREE: RESEARCH METHODOLOGY 12
3.1 Requirements Gathering Approach 12
3.2 Requirement Analysis 12
3.2.1 Functional Requirements 12
3.2.2 Non-Functional Requirements 13

3.3 System Design 14
3.3.1 System Architecture 14
3.3.2 Use Case Design 15
3.3.3 Database Design 16
3.3.4 Sign-in Design 18
3.4 Feature Selection 18
3.5 Experimental Setup 19
3.5.1 Development Tools and Environment 19
3.5.2 Development Methodology 19
3.5.3 Testing Strategy 20
3.5.4 Evaluation Criteria 20
3.5.5 Statistical Analysis 21

CHAPTER FOUR: EXPERIMENTAL RESULT AND DISCUSSION 22
4.1 Introduction 22
4.2 Overview of the Implemented System 22
4.3 Results of Feature Implementation 23
4.3.1 Authentication and Onboarding 23
4.3.2 Academic Management 24
4.3.3 Quiz and Past-Paper Features 25
4.3.4 AI Assistant and Tutor 26
4.3.5 Campus-Life Features 26
4.4 Automated Testing Results 27
4.4.1 Summary of Test Results 28
4.4.2 Flutter Automated Tests 28
4.4.3 Backend Unit Tests 29
4.4.4 Database Integration Tests 29
4.4.5 API Smoke Tests 29
4.4.6 Static Analysis 30
4.4.7 Defects Found by Running the Application 30
4.5 Evaluation Against Project Objectives 30
4.6 Discussion of Findings 31
4.6.1 Integration of Academic and Campus Services 31
4.6.2 Consistency of Academic Calculations 32
4.6.3 AI Assistance and Fallback Behaviour 32
4.6.4 Reliability of Notification and Sign-in Integrations 32
4.6.5 Local-First Operation and Synchronisation 33

4.6.6 Technical Quality and Test Coverage 33
4.7 Outstanding Issues and Deployment Readiness 34
4.8 Limitations of the Evaluation 35
4.9 Chapter Summary 35

CHAPTER FIVE: CONCLUDING REMARKS 37
5.1 Summary of the Study 37
5.2 Contribution of the Study 37
5.3 Limitations 38
5.4 Recommendations for Future Work 39
5.5 Conclusion 40

REFERENCES 41

LIST OF FIGURES

Figure 3.1: System architecture of Campus Companion 15
Figure 3.2: Use case diagram 16
Figure 3.3: Entity-relationship diagram of the Campus Companion database 17
Figure 3.4: Google Sign-In sequence 18
Figure 4.1: Onboarding and sign-in 24
Figure 4.2: Academic features 25
Figure 4.3: Campus-life features 27

LIST OF TABLES

Table 2.1: Summary of existing systems and related works 10
Table 3.1: Functional requirements 12
Table 3.2: Non-functional requirements 13
Table 3.3: System architecture layers 14
Table 3.4: Database tables in the Campus Companion schema 16
Table 3.5: Development tools and environment 19
Table 4.1: Summary of automated testing results 28
Table 4.2: Assessment of the specific objectives 31
Table 4.3: Outstanding issues and required verification 34

LIST OF ABBREVIATIONS

API	Application Programming Interface
APK	Android Package (Kit)
CRUD	Create, Read, Update, Delete
FCM	Firebase Cloud Messaging
GPA	Grade Point Average
ICS	iCalendar file format (.ics)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
REST	Representational State Transfer
SQL	Structured Query Language
UG	University of Ghana
UI/UX	User Interface / User Experience

CHAPTER ONE

INTRODUCTION

1.1 Background and Motivation

University students in Ghana manage a wide range of day-to-day academic and campus-life tasks: tracking lecture and examination timetables, monitoring grades and cumulative grade point average (GPA), keeping up with departmental and institutional announcements, finding their way around campus, registering for events and student organisations, and revising for quizzes and tests. In practice, these tasks are typically spread across several disconnected tools — paper diaries, generic calendar applications, printed noticeboards, WhatsApp or other social-media groups, spreadsheets for grade tracking, and informal word-of-mouth for locating buildings or offices. None of these tools is designed specifically for the structure of a Ghanaian tertiary institution, and none of them is aware of institution-specific conventions such as the University of Ghana's letter-grade and GPA scale.

At the same time, smartphone ownership among Ghanaian university students is now near-universal, and cross-platform mobile frameworks such as Flutter make it possible to build a single application that behaves consistently across Android and web targets without maintaining separate native codebases. This convergence of a clear, everyday student need and mature, accessible mobile-development tooling motivated the present project: an attempt to bring the scattered academic and campus-life tools students already improvise with into one coherent, purpose-built mobile application.

The motivation is reinforced by the observation that most existing “student app” examples available internationally assume grading scales, semester structures, and campus services that do not match the Ghanaian tertiary context. A Ghanaian student's GPA, for instance, is computed on a scale where a mark of 45-49 is graded E and does not count as a pass, which differs from scales used elsewhere. An application that hard-codes a different scale, or that treats campus services generically, is of limited practical use to the students it is meant to serve.

1.2 Statement of Problem

Ghanaian university students currently have no single, verified application that brings together the routine academic and campus-life tasks described above in a way that reflects the specific conventions of their institutions. The consequences of this fragmentation are threefold. First,

information is duplicated or lost: a class timetable kept on paper is not synchronised with a reminder app, and an announcement posted on a noticeboard may never reach a student who does not pass that location. Second, grade and GPA tracking is error-prone when done manually or with generic tools that do not encode the correct local grading bands, which can lead a student to misjudge their standing — for example, treating a borderline mark as a pass when, on the University of Ghana scale, it is not. Third, there is no single place where a student can get a quick, campus-aware answer to an everyday question ("when is my next class?", "what is on this week?", "where is this building?") without separately consulting several different sources.

This project addresses that gap by designing and building Campus Companion, an integrated mobile application and supporting backend service that unifies timetabling, examinations, grade/GPA analytics on the correct local scale, campus announcements and events, campus-location lookup, clubs and societies, quizzes, a digital student identity card, and an AI-assisted virtual assistant, behind a single authenticated account.

1.3 Scope of the Study

The study covers the design, implementation and testing of both halves of the Campus Companion system: a Flutter-based mobile client and a Node.js/Express REST API backed by a PostgreSQL database. The following areas fall within the scope of the study:

- Account registration, authentication (including Google Sign-In) and role-based access for three roles — student, lecturer and administrator.
- A local-first timetable and examination module with reminders and calendar (.ics) import/export, independent of server connectivity.
- A grade-entry and GPA/CWA analytics module that implements the University of Ghana grading scale, including honours-class and target-grade prediction.
- Campus announcements, events (creation, registration, cancellation and deletion with ownership rules) and a clubs/societies directory with join/leave functionality.
- A campus-location directory with a custom map view.
- A quiz module, including automated question extraction from past-paper PDFs using a large language model (LLM).

- An AI assistant/tutor that answers academic and campus questions, backed by a cloud LLM (Gemini) where configured and a local rule-based engine otherwise, designed to keep answering even when parts of the backend are unreachable.
- A digital student identification card with a scannable QR code.
- An administrator panel for provisioning lecturer and administrator accounts.
- Push-notification delivery (Firebase Cloud Messaging) for announcements, built and tested but intentionally left disabled until a production Firebase project is fully provisioned.

The following are explicitly out of scope: a real payment gateway for hall/students'-association dues (the "Student Hub" dues and shuttle features are demonstrated with static, device-local data only); native iOS support (the mobile client targets Android and web); and full production deployment of push notifications and Google Sign-In, both of which are implemented and tested end-to-end against development credentials but are pending a small number of remaining console/configuration steps outside the application code itself. These limitations are revisited in Chapter Five.

1.4 Research Objectives

1.4.1 Global Objective

The global objective of this study is to design, implement and evaluate an integrated, mobile-first application that consolidates academic scheduling, grade analytics and campus-life services for Ghanaian tertiary students within a single secure platform, reflecting the specific academic conventions of their institutions.

1.4.2 Specific Objectives

1. To design and implement a cross-platform mobile client (Flutter) and a RESTful backend service (Node.js/Express with PostgreSQL) providing unified, role-based authentication for student, lecturer and administrator accounts, including secure third-party sign-in via Google.
2. To implement a local-first academic timetable, examinations and alerts module supporting calendar (.ics) import/export and reminder notifications without depending on constant server connectivity.

3. To implement a grade and GPA/CWA analytics module that correctly applies the University of Ghana grading scale, keeping the mobile client and backend in agreement through a single shared source of truth.
4. To implement campus-life features — events, clubs and societies, campus-location lookup, quizzes and a digital student identification card — behind the same authenticated platform.
5. To design and implement an AI-assisted virtual assistant that answers academic and campus questions using a cloud large language model where available, falls back to a local rule engine otherwise, and remains useful when individual backend data sources are unreachable.
6. To integrate push-notification delivery via Firebase Cloud Messaging for campus announcements.
7. To verify the correctness of the resulting system through a layered automated-testing strategy — offline unit tests, database/integration tests, and end-to-end smoke tests — and through deliberate mutation testing of key business logic.

1.5 Research Contribution

This study contributes an integrated application architecture that brings academic-scheduling and campus-life functionality together under the grading and institutional conventions specific to Ghanaian tertiary education — a combination not offered by generic, internationally-oriented student-planner applications. It further contributes a concrete design pattern for building an AI assistant that degrades gracefully: rather than failing an entire query when one backend data source is unavailable, the assistant answers from whichever sources it can reach and reports, source by source, which ones it could not. Finally, the study contributes a demonstrated test-driven development discipline for a full-stack student application — offline unit tests, database-backed integration tests, end-to-end smoke tests, and deliberate mutation testing to confirm that the test suite actually detects regressions — together with a shared, machine-readable fixture (the grading-scale definition) used to keep the mobile client and the backend service from silently drifting apart on a rule as consequential as how a GPA is computed.

1.6 Organization of the Study

The remainder of this study is organised as follows. Chapter Two reviews literature relevant to mobile academic-management and campus-life applications, authentication and push-notification patterns in mobile systems, and AI-assisted educational assistants. Chapter Three describes the research methodology: how the requirements were gathered and analysed, the system's design (architecture, use cases, database and sign-in flow), the features selected, and the development and testing approach used to build and validate it. Chapter Four presents and discusses the results of that validation, including test outcomes and an assessment of the implemented features against the objectives in Section 1.4. Chapter Five concludes the study, summarising its contributions, acknowledging its limitations, and recommending directions for future work.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter reviews literature relevant to the design of Campus Companion. It is organised around the system's main technical and functional concerns: mobile applications built to support students' academic and campus life (Section 2.2); mobile and smartphone use among Ghanaian university students, which motivates and contextualises this study (Section 2.3); authentication and security practices for mobile applications and the standards behind them, relevant to the system's sign-in design (Section 2.4); push-notification architectures, relevant to announcement delivery (Section 2.5); and AI assistants and chatbots in education, relevant to the system's AI assistant and tutor (Section 2.6). Section 2.7 compares existing systems with the one built in this study, and Section 2.8 summarises the chapter and identifies the gap this study addresses.

2.2 Mobile Applications for Academic Scheduling and Student Life Management

A recurring theme in the literature on student-facing mobile applications is the attempt to bring separate parts of a student's life — tasks and deadlines, wellbeing, and campus services — into a single tool. Zhao and Pardo (2017) describe the user-centred design of myUniMate, an application that lets first-year students track and reflect on several aspects of their university life. Its components include a to-do list with reminders, a calendar view of those tasks, mood tracking, user-defined measures, and simple charts of the tracked information. In a four-week field trial with 11 first-year engineering students, the students' perceived competence and relatedness support and their learning motivation increased, while the autonomy support they attributed to the application decreased. myUniMate is a personal organiser: it does not hold a class timetable or grades, and it offers no campus-wide services.

Baras et al. (2018) describe an application that keeps a student's schedules and deadlines in one place and adds virtual-tutor features, with a particular focus on supporting students' mental health alongside their academic success. Like myUniMate, it centres on the individual student's organisation rather than on campus-wide information.

Wang and Yan (2018) take the other direction: they describe the design and implementation of an Android “mobile smart campus” system that delivers a campus's internet services through

one mobile application. This is closer in spirit to the campus-life half of Campus Companion — events, clubs, campus locations and announcements — than to its academic half. The two strands of literature, personal academic organisers on one hand and campus-wide service systems on the other, are rarely combined into one evaluated system in the papers surveyed here; that combination, a single authenticated application spanning both a student's own academic data and campus-wide services, is one gap this study positions itself against (Section 2.8).

A methodological point carries across this literature: the systems reviewed are typically evaluated with small, single-institution samples over short periods (myUniMate's trial involved 11 students for four weeks), and none reports a long-running production deployment. This is consistent with the practical reality that student-facing academic tools are frequently built and evaluated as single-institution or final-year projects rather than at commercial scale, which is also the context of the present study.

2.3 Mobile Learning and Smartphone Use Among Ghanaian University Students

Ghana-specific studies show that mobile devices are already part of Ghanaian university students' academic lives, but largely outside formal academic systems. Afful and Boateng (2023) studied distance-education students at the University of Ghana and the University of Education, Winneba, using a decomposed Theory of Planned Behaviour and an explanatory sequential mixed-methods design to examine how students' attitudinal, social and control beliefs affect their mobile learning. They found that students mostly used their phones for calls and text messages, and recommended training in retrieving information on mobile devices. Adzifome and Agyei (2022), studying students at a Ghanaian university, reported that the facilities needed for mobile learning were largely available and students' experiences of it were positive, but that much of this mobile learning took place outside the institution's learning management system.

These findings are directly relevant to the problem motivating the present study (Section 1.2): Ghanaian students have mobile devices and use them for study, but the tools they use are general-purpose ones that are not built around their institution's academic structure (grading scale, semester calendar, courses and clubs). Neither study evaluates a purpose-built, institution-aware application; their findings describe the gap such an application would need to fill. Campus Companion's institution-aware design — in particular, its implementation of

the University of Ghana grading scale rather than a generic one (Sections 1.2 and 3.3.3) — responds directly to this gap.

2.4 Authentication and Security in Mobile Applications

Because Campus Companion authenticates three distinct roles (student, lecturer, administrator) and supports both a password-based flow and third-party sign-in through Google, the security of its authentication design draws on the mobile-OAuth literature. Chen et al. (2014), in a widely cited empirical study, examined how 149 mobile applications implemented OAuth and found that 89 of them (59.7%) were implemented incorrectly and were therefore vulnerable — commonly by trusting client-side information as if it had been verified by the server. Their central lesson, that the application's own backend, not the mobile client, must verify any identity token before trusting it, is the pattern Campus Companion's backend follows: the Google ID token obtained on the mobile client is exchanged for a Firebase ID token, which the backend independently verifies through the Firebase Admin SDK before issuing its own session token, rather than trusting any claim the mobile client makes about who the user is (Figure 3.4).

Several standards underpin this design. Google Sign-In builds on OAuth 2.0, the authorisation framework whose misuse Chen et al. (2014) document (Hardt, 2012). After sign-in, the backend issues its own session token as a JSON Web Token: a compact, digitally signed set of claims that the server can check on every request without keeping session state (Jones et al., 2015). That suits the REST architectural style described by Fielding (2000), in which each request carries everything the server needs to process it. Passwords are never stored; the backend keeps only a bcrypt hash, a salted and deliberately slow hash whose cost can be raised as computers become faster (Provos & Mazières, 1999). These choices respond to the OWASP Top 10, which ranks broken access control as the most serious web application security risk and lists identification and authentication failures among the others (OWASP Foundation, 2021).

This body of work underlines a general principle followed throughout Campus Companion's design: role and identity claims are only trusted when they come from a server-side verification step, never from client-supplied data. For instance, the backend enforces that a self-registering user can only ever be created with the student role, regardless of any role value the client might send — precisely the class of client-trust vulnerability this literature warns against.

2.5 Push Notification Architectures in Mobile Systems

Campus Companion's announcement-delivery design uses Firebase Cloud Messaging (FCM), the push-delivery service most commonly used in the Android ecosystem. Samarin et al. (2024) provide the most directly relevant recent study of FCM in practice: examining 21 popular Android secure-messaging applications that use FCM, they found that 11 leaked some form of user-identifying metadata through push notifications and 4 leaked actual message content, and that none of the affected applications disclosed this in its privacy policy.

This finding shaped a design decision in Campus Companion: the FCM payload used for announcement notifications carries no announcement content, only a signal that new content is available, which the client then fetches from the authenticated announcements endpoint. Keeping content out of the payload avoids the class of leak Samarin et al. (2024) document, at the cost of one extra request when the notification is opened.

2.6 AI-Assisted Virtual Assistants and Chatbots in Education

Two systematic reviews give a broad picture of chatbot use in education. Okonkwo and Ade-Ibijola (2021), reviewing 53 articles, identify educational chatbots used for teaching, administration, assessment, advisory work and research; the benefits reported include immediate help for students, and the challenges include ethical, programming and maintenance issues. Wollny et al. (2021), reviewing 74 publications drawn from an initial pool of 2,678, frame the field's open research challenges around chatbot evaluation, mentoring and adaptation to the learner.

A study set directly in the Ghanaian context, and therefore the most relevant single source for this project's AI assistant, is Essel et al. (2022). The authors built KNUSTbot, a chatbot deployed in a multimedia-programming course at Kwame Nkrumah University of Science and Technology, motivated by the high instructor-to-student ratio common in Ghanaian higher education: a chatbot can give large cohorts immediate answers to routine questions that would otherwise wait for an instructor. In a pretest–posttest study with 68 undergraduates allocated across a 2×2 experimental design, students who interacted with the chatbot performed better academically than those who interacted with the instructor.

Essel et al. (2022)'s result is encouraging for the inclusion of an AI assistant in Campus Companion, but its sample (68 students, one course, one institution) is small, so the result should be read as a promising single data point rather than settled evidence that any chatbot

improves outcomes. The studies reviewed here also focus on whether chatbots help learning, not on how a chatbot should behave when a service it depends on is unavailable. Campus Companion's AI assistant draws on several independent backend data sources (grades, events, alerts, campus locations) plus device-local data (today's classes and exams), and was designed to treat the failure of any single source as something to report to the student rather than a reason to fail every answer.

2.7 Review of Existing Systems

Table 2.1 compares Campus Companion with existing systems a Ghanaian student might use and with the related systems reviewed in this chapter.

Table 2.1: Summary of existing systems and related works

System / work	Description	Strengths	Limitations for this context
University of Ghana Sakai LMS (University of Ghana, n.d.)	The University's learning management system for course materials, assignments and notices.	Official; used across the University's courses.	Centred on course delivery; no GPA analytics on the UG scale, events, clubs or campus map.
MyStudyLife (MyStudyLife, n.d.)	Commercial student planner for timetables (weekly or rotating), exams, tasks and reminders, with grade tracking; syncs across devices and works offline.	Mature, multi-platform, usable offline.	Not tied to any institution, so it has no institution-specific grading rules or campus services; some features need a paid subscription.
myUniMate (Zhao & Pardo, 2017)	Research prototype for first-year students: to-do list with reminders, mood tracking and user-defined measures.	Designed and evaluated with students.	Personal organiser only; no timetable, grades or campus services; 11-student trial.
Mobile smart campus (Wang & Yan, 2018)	Android system delivering a campus's internet services through one application.	Brings campus services into one app.	Campus services only; no personal academic data.
KNUSTbot (Essel et al., 2022)	Chatbot teaching assistant for one course at a Ghanaian university.	Ghanaian context; students using it performed better.	A single-course chatbot, not an integrated campus application.
Campus Companion (this study)	Mobile application combining timetable, exams, UG-scale GPA analytics, announcements, events, clubs, campus map, quizzes, an AI assistant and a digital ID.	Personal academic data and campus services behind one account; institution-aware grading.	Push notifications and Google Sign-In await configuration; not yet evaluated with users (Section 5.3).

No single system in Table 2.1 combines a student's own academic data, institution-specific grading, campus-wide services and an AI assistant in one application, which is the combination this study builds.

2.8 Summary and Research Gap

Taken together, the literature reviewed in this chapter establishes three things relevant to this study. First, mobile applications that organise a student's personal academic life are a well-studied pattern (Zhao & Pardo, 2017; Baras et al., 2018), as are applications that bring campus-wide services together (Wang & Yan, 2018) — but the two are rarely combined and evaluated as one system behind a single authenticated account, which is the combination Campus Companion attempts (Table 2.1). Second, Ghanaian students already use mobile devices for study, largely outside formal academic systems (Afful & Boateng, 2023; Adzifome & Agyei, 2022), but no purpose-built, institution-aware application — one that, for instance, implements the correct local grading scale — is evaluated in this literature; this is the specific gap Section 1.2 identifies and this study addresses. Third, while mobile authentication security (Chen et al., 2014) and push-notification privacy (Samarin et al., 2024) are each well studied, and AI chatbots in education show early promise in a Ghanaian setting (Essel et al., 2022), none of the systems reviewed combines these concerns within a single academic-and-campus-life application. Chapter Three describes how Campus Companion was designed and built to close this gap.

CHAPTER THREE

RESEARCH METHODOLOGY

This chapter describes how Campus Companion was specified, designed, built and evaluated. Because this is a software development project rather than a data-modelling study, the template's methodology headings (dataset description, preprocessing, oversampling, feature selection, model training and statistical tests) have been adapted to the corresponding stages of building and evaluating a software system: gathering and analysing requirements, designing the system, selecting its features, and setting up its development and testing.

3.1 Requirements Gathering Approach

The requirements for Campus Companion were identified primarily through personal analysis of the academic and campus-life needs of university students. The requirements-gathering process involved examining common challenges associated with accessing academic information, organising study activities, locating campus resources and keeping up with university announcements and events.

Based on this analysis, the functional and non-functional requirements of the application were established. These requirements informed the design and development of the system's major modules, including timetable management, examination support, grade tracking, quizzes, campus information, events, clubs and the AI-powered assistant.

No formal interviews or questionnaire-based surveys were conducted as part of the requirements-gathering process. Consequently, the requirements presented in this study reflect the developer's analysis and design decisions rather than findings from a formal participant study.

3.2 Requirement Analysis

The gathered requirements were organised into functional requirements, which state what the system must do, and non-functional requirements, which state the qualities it must have (Sommerville, 2016). Tables 3.1 and 3.2 list them; Chapter Four checks the implemented system against them.

3.2.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement	Users
FR1	The system shall let a student register with a name, email address and password; self-registration always creates a student account.	Student
FR2	The system shall authenticate users by email and password, and by Google Sign-In once that provider is enabled.	All users
FR3	The system shall let an administrator create lecturer and administrator accounts.	Administrator
FR4	The system shall let a user choose their institution, which sets the grading scale and branding the app uses.	All users
FR5	The system shall let a student add, edit, delete, import (.ics or CSV) and export (.ics) timetable classes, stored on the device, with class reminders.	Student
FR6	The system shall let a student record exams, see countdowns and receive exam reminders.	Student
FR7	The system shall compute GPA or CWA from the student's grades on the institution's scale, with honours class and target-class prediction.	Student
FR8	The system shall show campus announcements and let an administrator post them, with push notifications once push delivery is enabled.	All users, Administrator
FR9	The system shall let users browse and RSVP to events; lecturers and administrators create events; a lecturer deletes only their own events, an administrator any event.	All users, Lecturer, Administrator
FR10	The system shall let users search clubs and join or leave them.	Student
FR11	The system shall list campus locations by category, show them on a map, and open them in a maps app.	All users
FR12	The system shall let lecturers and administrators create quizzes, including from a past-paper PDF, and let students take them and see their scores.	Lecturer, Administrator, Student
FR13	The system shall answer academic and campus questions through an AI assistant and tutor, and keep answering when some backend data sources are unreachable.	Student
FR14	The system shall display a digital student ID card with a QR code.	Student

3.2.2 Non-Functional Requirements

Table 3.2: Non-functional requirements

Requirement	Description
Security	Passwords are hashed with bcrypt before storage; every protected request needs a valid JSON Web Token; roles are enforced on the server, not only hidden in the interface; request bodies are validated before use.
Privacy	Push notifications carry no announcement content (Section 2.5); the optional profile details for the ID card stay on the device.
Resilience	The timetable and exams work without the server; the AI assistant reports which data sources it could not reach instead of failing outright.
Correctness	The grading scale is defined once and shared by the server and the app, so the two cannot disagree on a grade.
Usability	Core features are reachable from the dashboard through labelled shortcuts, and forms show their errors in place.
Portability	One Flutter codebase runs on Android and the web, against one REST API.
Maintainability	The code is layered (screens, state, repositories; routes, controllers, services) and covered by automated tests at three levels (Section 3.5.3).

3.3 System Design

3.3.1 System Architecture

Campus Companion follows a layered client–server architecture, summarised in Table 3.3 and shown in Figure 3.1.

Table 3.3: System architecture layers

Layer	Technology	Responsibility
Mobile client	Flutter, Riverpod (state management), Dio (HTTP)	Renders the UI; holds local-first data (timetable, exams) on-device; calls the REST API for server-backed features.
Backend API	Node.js, Express, Zod (validation)	Stateless REST API; authentication/authorisation, business rules (grading scale, event-deletion ownership), request validation.
Database	PostgreSQL	Durable relational storage for the entities in Table 3.4.
Identity provider	Firebase Authentication	Issues and verifies Google Sign-In identity tokens, shared with the push-notification Firebase app.
Push delivery	Firebase Cloud Messaging	Delivers announcement notifications to registered device tokens; payload carries no announcement content (Section 2.5).
AI assistant backend	Google Gemini API (optional), local rule engine (fallback)	Answers assistant/tutor queries; falls back to an offline rule engine when no API key is configured or the call fails.

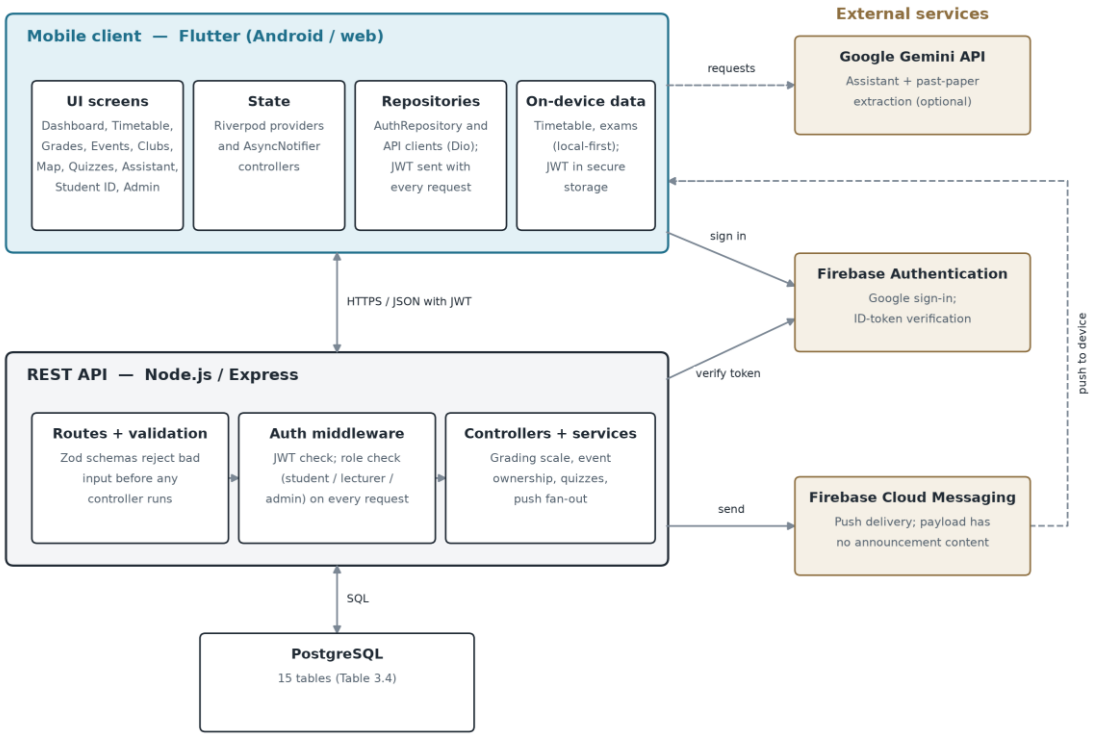


Figure 3.1: System architecture of Campus Companion

Each layer communicates over well-defined interfaces: the mobile client never queries the database directly, and the backend never assumes a specific client — the same REST API serves the Flutter app regardless of whether it runs on Android or as a web build. Role-based access control (student, lecturer, administrator) is enforced in the backend on every request that needs it, not merely hidden in the mobile UI, so that a client cannot grant itself privilege by skipping a UI check (Section 2.4).

3.3.2 Use Case Design

Figure 3.2 shows the system's three actors and what each can do. Students register themselves and use the academic and campus-life features. Lecturer and administrator accounts cannot be self-registered: an administrator creates them (FR3). Lecturers create events and quizzes and may delete only the events they created; administrators can also delete any event or quiz, post announcements, record grades and create staff accounts. These rules are enforced by the backend's role checks, and the diagram was drawn from those checks rather than from the interface.

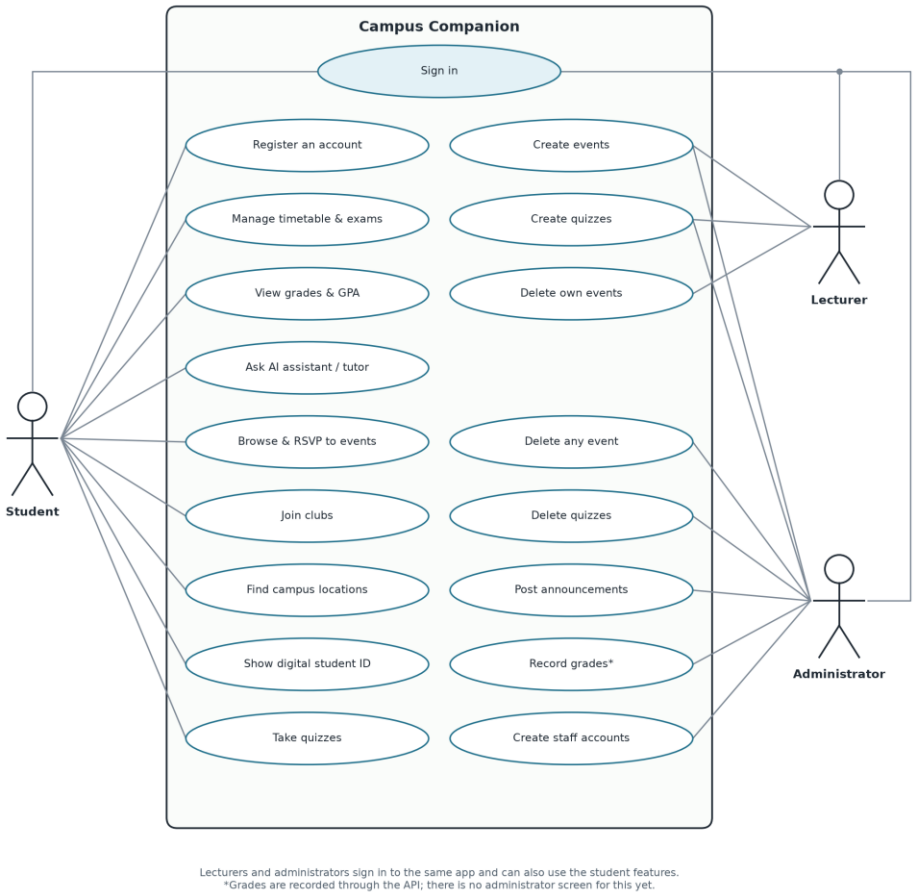


Figure 3.2: Use case diagram

3.3.3 Database Design

The system's data is held in a PostgreSQL database of fifteen tables (`backend/db/schema.sql`), listed in Table 3.4; Figure 3.3 shows how they relate.

Table 3.4: Database tables in the Campus Companion schema

Table	Purpose
users	One row per account (student, lecturer or admin), holding the password hash and/or a linked Google UID, role, and push preferences.
courses	Canonical course records (code, title, lecturer, credit hours), referenced by schedules, grades and quizzes to avoid free-text duplication.
schedules	A user's timetabled class slots (day, start/end time, room), used server-side only as an optional cloud-backup layer — the app itself is local-first (Section 1.3).
grades	A score per user per course per semester; the letter grade is a database-generated column derived from the score using the University of Ghana scale, so it cannot drift from the score it is derived from.
clubs / club_memberships	Club records and the many-to-many membership junction between users and clubs.
events / event_registrations	Campus events (optionally linked to a club and to the user who created them) and the many-to-many registration junction between users and events.

Table	Purpose
announcements	Broadcast messages posted by staff, categorised (general, academic, emergency) and timestamped for sync.
quizzes / quiz_questions / quiz_attempts / quiz_attempt_answers	A course-anchored quiz, its questions (multiple-choice or open-ended), and each student's attempt and per-question answers.
device_tokens	One row per installed app instance registered for push notifications, keyed by a unique FCM token.
locations	Named campus points of interest with a category and coordinates, used by the campus-map feature.

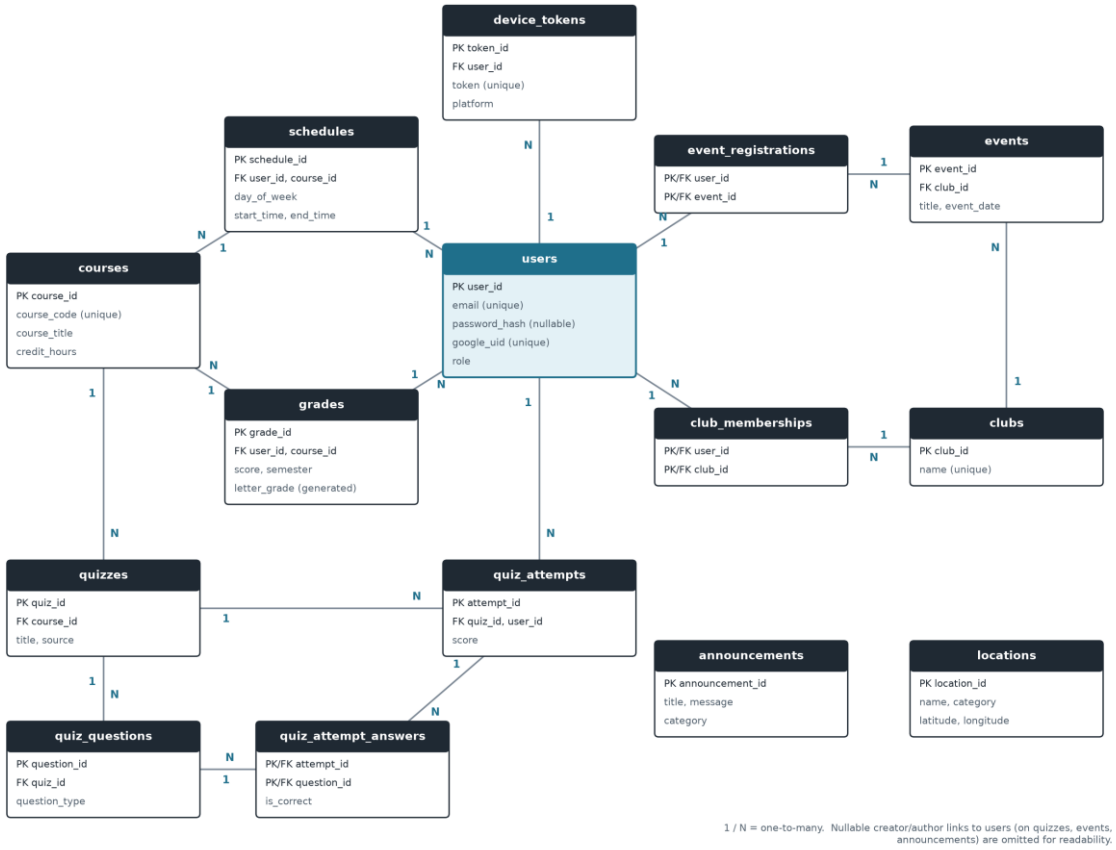


Figure 3.3: Entity-relationship diagram of the Campus Companion database

Data integrity is enforced before and at the database. Incoming API request bodies are validated and coerced against a schema (the `zod` library) before any controller logic runs — for example, `registerSchema` trims and lower-cases an email address and enforces a minimum password length before a new user row is ever written (`backend/src/controllers/authController.js`). A unique index on `LOWER(email)` guarantees case-insensitive matching regardless of how a client submits an address. The letter grade is derived from the numeric score inside the database itself (a `GENERATED ALWAYS AS` column), rather than accepted as separate client input that could disagree with its score.

Externally authored calendar data (`.ics`/CSV files a student imports into their local timetable) is parsed and normalised into the app's internal schedule model by a dedicated, tested parser before any other feature uses it.

3.3.4 Sign-in Design

Figure 3.4 traces one request path through the layers in detail: Google Sign-In. The mobile app never tells the backend who the user is. It forwards a Firebase ID token, the backend verifies that token with Firebase itself, and only then looks up, links, or creates the account and issues the app's own session token (JWT). This is the server-side verification pattern recommended by Chen et al. (2014), discussed in Section 2.4.

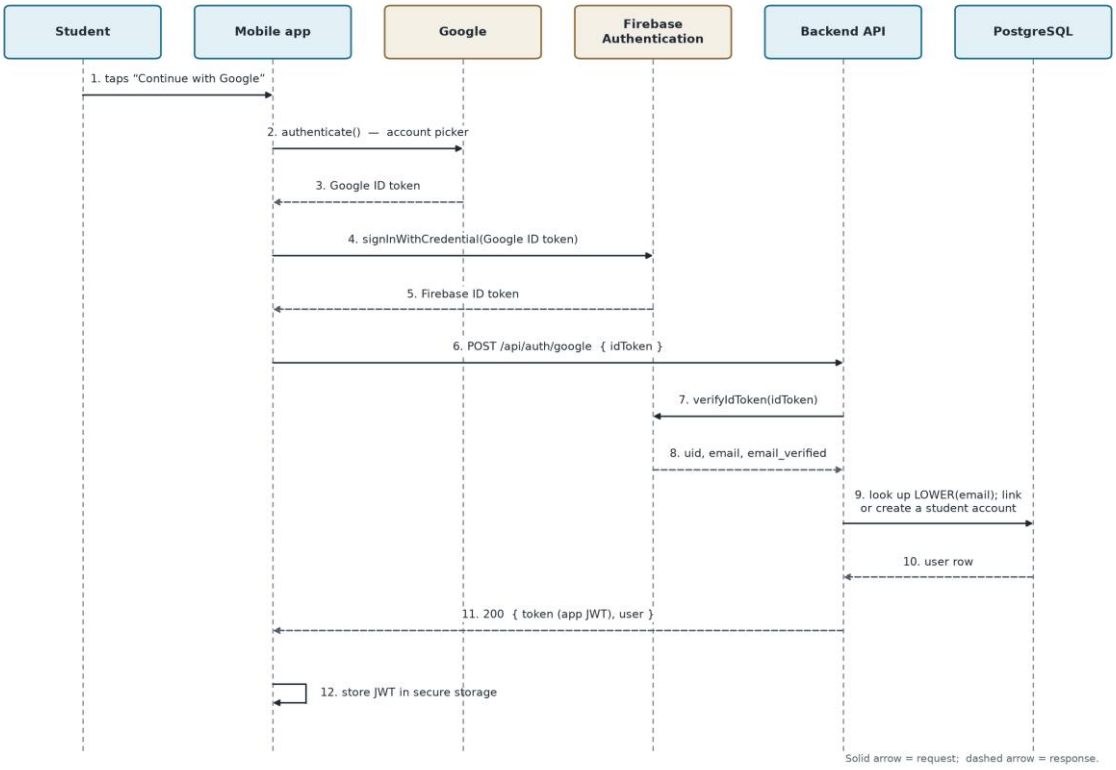


Figure 3.4: Google Sign-In sequence

3.4 Feature Selection

The features were selected from the requirements in Section 3.2, weighed against the time available, external dependencies, and relevance to the problem in Section 1.2; Section 1.3 sets out the resulting scope. Scoping was revisited iteratively rather than fixed up front. Between an administrator management panel and push notifications, the administrator panel was built first because it needed no external service. Push notifications and Google Sign-In were built and tested but deliberately left disabled pending Firebase console configuration outside the

application's own code (PROJECT_AUDIT.md, Sections 2–3). The Student Hub's dues, shuttle and lost-and-found screens were deliberately kept on static, device-local demonstration data rather than integrating a real payment gateway, which was judged out of scope for the time available (Section 1.3). Apple sign-in and a guest mode appear on the sign-in screens but are not yet implemented (Section 5.3).

3.5 Experimental Setup

3.5.1 Development Tools and Environment

Table 3.5: Development tools and environment

Category	Tool / technology	Purpose
Mobile client	Flutter (Dart), Riverpod, Dio	Cross-platform app for Android and web; state management; HTTP client
Backend	Node.js, Express, Zod, bcryptjs, jsonwebtoken	REST API; request validation; password hashing; session tokens
Database	PostgreSQL	Relational storage (Table 3.4)
External services	Firebase Authentication and Cloud Messaging (firebase-admin); Google Gemini API	Google Sign-In; push notifications; AI assistant and past-paper extraction
Testing	flutter_test, node:test, end-to-end smoke script	Automated tests at three levels (Section 3.5.3)
Tools	Visual Studio Code, Android Studio and Android emulator, Git, Google Chrome	Editing; Android builds and emulation; version control; web testing
Deployment	Render (render.yaml)	Hosting configuration for the API, web build and database
Hardware	Windows 11 laptop, 7.7 GB RAM	Development and testing

3.5.2 Development Methodology

The system was built iteratively and incrementally, in the spirit of Agile development (Beck et al., 2001): working software was produced feature by feature rather than in large, infrequent releases. For each feature, the backend route(s) and database schema were implemented and tested first, followed by the corresponding mobile screens, state-management providers and widget tests — so that a feature's server-side contract existed and was verified before client code was written against it. Within the mobile client, a repository pattern separates API/storage access (e.g. `AuthRepository`, wrapping an `AuthApi` and a token store) from UI state, which is held in Riverpod `AsyncNotifier`/`StateProvider` controllers; this keeps a screen's widgets free of networking logic and makes the underlying repository replaceable with a fake in tests. A shared-fixture pattern guards rules that both the backend and the mobile client must agree on: the University of Ghana grading scale is defined once, in `docs/grading-scale-ug.json`, and both the backend's test suite and the Flutter client's parity test read that same file,

so the two codebases cannot silently drift onto different grading rules (Section 2.3's finding that institution-specific conventions are often missed from generic tools was a direct motivation for keeping this rule in one place rather than duplicating it). A local-first pattern was used for data the student owns and edits constantly — the timetable and exams live primarily on the device, with the server's equivalent routes kept working as an unused, optional backup layer, because routing the dashboard through the server's (normally empty) timetable was found, during development, to risk replacing a student's real classes with nothing.

3.5.3 Testing Strategy

Testing is layered into three levels (Sommerville, 2016). Offline unit tests (`node:test` on the backend, `flutter test` on the mobile client) run without a network connection or database and check pure logic — the grading-scale calculations, the push fan-out logic against in-memory fakes, and individual screens' behaviour. Database/integration tests run the same logic against a real local PostgreSQL instance to catch anything a fake database could hide, such as a uniqueness constraint violation. End-to-end smoke tests run last, against a running API and database, and walk realistic user journeys (registration, login, creating and registering for an event, taking a quiz) the way a real client would, rather than calling internal functions directly.

The tests themselves were checked by mutation testing: for business logic judged important enough to double-check (event-deletion ownership, self-registration never granting an elevated role, the session-expiry flag, the Google sign-in cancel path), the real logic was deliberately broken, the relevant tests were confirmed to fail, and the code was then restored and re-verified. A test suite that cannot detect a deliberately introduced bug is not validating anything, so this step was treated as necessary rather than optional.

3.5.4 Evaluation Criteria

The system was evaluated against five criteria: functional correctness (the automated tests in Section 3.5.3 pass); objective coverage (each specific objective in Section 1.4.2 is traced to evidence, Table 4.2); security effectiveness (role enforcement and self-registration rules are tested, and Google sign-in tokens are verified against the live identity provider); resilience (offline and partial-failure behaviour is tested against simulated failures); and end-to-end behaviour (the application is run as a whole, not only screen by screen).

As verified on 24 October 2026 (PROJECT_AUDIT.md), the backend's offline unit suite passed 50/50, its database suite passed 14/14, and its end-to-end smoke suite passed 182/182;

the mobile client's `flutter test` suite passed 366/366 and `flutter analyze` reported no issues. Two checks were done manually rather than by an automated test: a request carrying a garbage Google sign-in token was sent to the live Firebase project to confirm it is rejected with 401 (invalid token) rather than 503 (feature not configured) — proving the credential itself is live and correctly wired, not merely that the code compiles — and a debug Android build was produced to confirm the push-notification and Google Sign-In packages compile and link correctly on-device. These figures are a point-in-time snapshot; Chapter Four re-verifies them freshly rather than repeating this snapshot as if it still held unchanged.

3.5.5 Statistical Analysis

The evaluation uses descriptive counts only — the number of tests and checks that pass or fail — because its question is whether the system behaves as specified, not whether one group or design performs better than another. No hypothesis is tested, so no inferential statistics (significance tests, confidence intervals or effect sizes) are reported. This study does not include a user survey; Chapter Five recommends one.

CHAPTER FOUR

EXPERIMENTAL RESULT AND DISCUSSION

4.1 Introduction

This chapter presents and discusses the implementation and testing results of Campus Companion, a mobile application designed to integrate academic management and campus-life services for university students in Ghana. The chapter examines the major features implemented, the outcomes of automated testing, the extent to which the project objectives have been achieved, and the limitations identified during development.

The evaluation is based on results recorded on 30 October 2026 from the Flutter mobile application tests, backend unit tests, database integration tests, API smoke tests and static analysis, together with defects found by running the complete application. These results provide evidence of the behaviour of the software under the tested conditions. They do not, however, establish that every feature has been validated in a production environment or that the application has improved students' academic performance.

4.2 Overview of the Implemented System

Campus Companion was developed as a Flutter mobile application supported by a Node.js and Express backend and a PostgreSQL database. It brings several academic and campus-related functions into one application.

The implemented features include authentication with role-based access, Google Sign-In, an academic dashboard, timetable management, examination information, grade calculations, academic analytics, quizzes, past-paper question extraction, an AI Assistant and Tutor, campus locations, events, clubs, announcements, and a digital identification card.

The implementation also includes local-first functionality for selected academic features, calendar export, local reminders, and an offline fallback for supported AI interactions. Firebase Cloud Messaging and Google Sign-In have been implemented, although successful push-notification delivery to a physical device remains to be verified and Google Sign-In awaits a final configuration step in the Firebase console.

The results indicate that Campus Companion has progressed beyond an interface-only demonstration. It contains functional mobile and backend components and has undergone

several forms of automated testing. Nevertheless, the maturity of individual modules varies, particularly where backend integration or real-device verification remains outstanding.

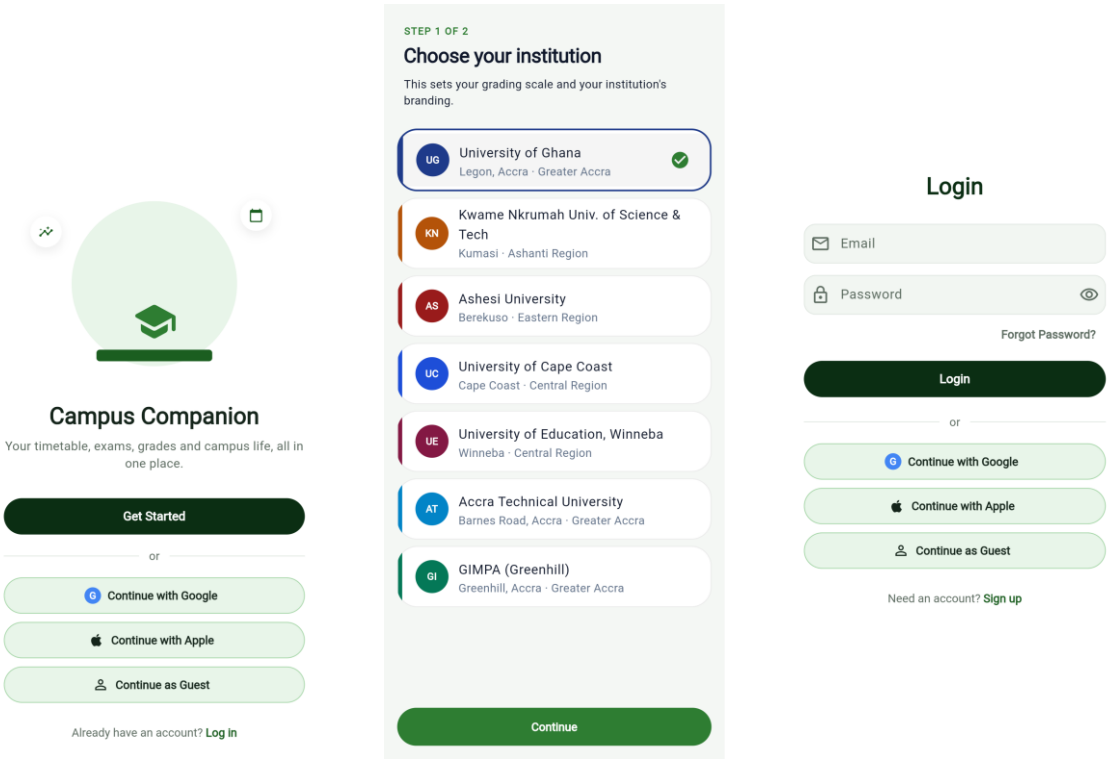
The screenshots in this chapter (Figures 4.1 to 4.3) show the application running against the local backend and database, signed in as the seeded demonstration student. The Android build was installed and launched on an emulator, but the development machine (7.7 GB of memory) could not keep the emulator responsive for a full walkthrough, so the screenshots were taken from the same codebase compiled as a web build and shown in a phone-sized (412 × 915) browser window. Flutter draws every screen from the same widget code on both targets, so the screens are the same apart from the Android status bar. The only values entered by hand are the fictional student ID, level and programme in the optional profile, which feed the digital identification card.

4.3 Results of Feature Implementation

4.3.1 Authentication and Onboarding

Users register and sign in with an email address and password; passwords are stored only as bcrypt hashes, and sessions use JSON Web Tokens. Three roles are supported: students register themselves, while lecturer and administrator accounts can only be created by an administrator. Role checks are enforced by the backend on every protected request, not merely hidden in the interface.

On first launch the student sees a Welcome screen, chooses an institution, which sets the grading scale and branding, and may add optional profile details for the digital identification card (Figure 4.1). Google Sign-In is implemented end to end and its backend verification has been checked against the live Firebase project, but it cannot sign anyone in until the Google provider is enabled in the Firebase console. Apple sign-in and a guest mode appear on the Welcome and Login screens but are not yet implemented; tapping either shows a short notice rather than doing nothing.



(a) Welcome (b) Institution picker (c) Login

Figure 4.1: Onboarding and sign-in

4.3.2 Academic Management

The academic management features were implemented to help students organise class schedules, prepare for examinations, and understand their academic results.

The timetable module displays schedules and supports local-first handling. Calendar export allows supported schedule information to be exported in the iCalendar format, while local notifications support reminders. Examination-related features provide access to examination information and preparation activities.

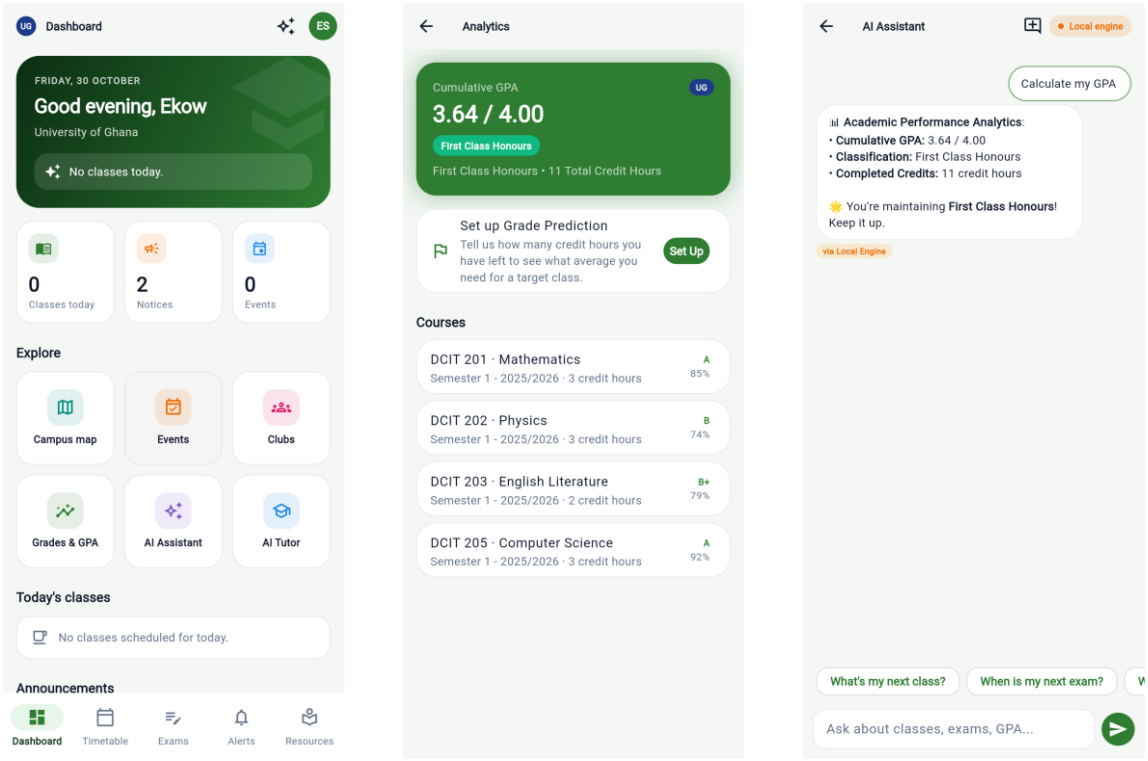
The grade management module calculates grades and grade point averages using the implemented University of Ghana grading scale. A grading inconsistency discovered during development was corrected so that the relevant mobile and backend calculations interpret supported score ranges consistently. Figure 4.2(b) shows the result for the demonstration student: a cumulative GPA of 3.64 (First Class Honours), the same figure the backend's grading tests expect for this student.

This correction demonstrates the importance of validating business rules across different components of the same application. If the interface displays a grade that differs from the grade

used in GPA calculations, users may receive misleading academic information. Consistent grading logic reduces this particular risk.

The application also provides academic summaries and target-class prediction. These features are intended to help students reason about possible academic outcomes. However, their predictive accuracy has not been established through testing against a representative dataset.

A limitation of the current timetable implementation is that it relies on device-local data as its primary source. The backend schedule route is not currently used by the application as a synchronised backup. As a result, complete cross-device timetable synchronisation has not been demonstrated.



(a) Dashboard (b) GPA analytics (c) AI Assistant

Figure 4.2: Academic features

4.3.3 Quiz and Past-Paper Features

The quiz module supports the creation and completion of quizzes. It also supports the import of past-paper PDFs and Gemini-assisted extraction of questions from supported documents.

The implementation prevents answer keys from being revealed before submission, and automated tests cover the relevant quiz and past-paper extraction behaviour.

These capabilities provide a foundation for examination preparation within the application. However, the quality of extracted questions may depend on document formatting, text clarity, and the structure of the original paper. Passing automated tests does not establish that every question will be extracted correctly from every PDF. Further evaluation using a varied collection of actual past papers would be required to measure extraction accuracy and identify document formats that require additional handling.

4.3.4 AI Assistant and Tutor

The AI Assistant provides a conversational interface for academic and campus-related questions. It uses Gemini when the necessary API configuration is available and has a local keyword-based fallback for supported interactions. Figure 4.2(c) shows the fallback answering "Calculate my GPA" from the student's stored grades; the reply is labelled "via Local Engine".

The assistant can retrieve relevant information from supported application areas, including timetable information, grades, campus locations, events and alerts. Context retrieval is designed so that failure to retrieve one category does not prevent the use of the other available categories; the assistant instead tells the student which information it could not reach. The AI Tutor reuses the same conversational interface for tutoring-related interactions.

The implementation therefore supports both external AI-assisted interactions and a more limited fallback mode. The fallback provides some resilience when the external service is unavailable, but it does not offer the same capabilities as a full generative AI model.

The current results establish that assistant-related logic is covered by automated testing. They do not establish the factual accuracy of every AI response, the quality of explanations across all subjects, or a measurable improvement in student learning, and the assistant has not yet been exercised against a live Gemini API key or on a physical device. Those outcomes require a separate evaluation using representative questions and, if educational effectiveness is being assessed, participation from intended users.

4.3.5 Campus-Life Features

Campus Companion includes campus locations, events, clubs, announcements, and digital identification features (Figure 4.3).

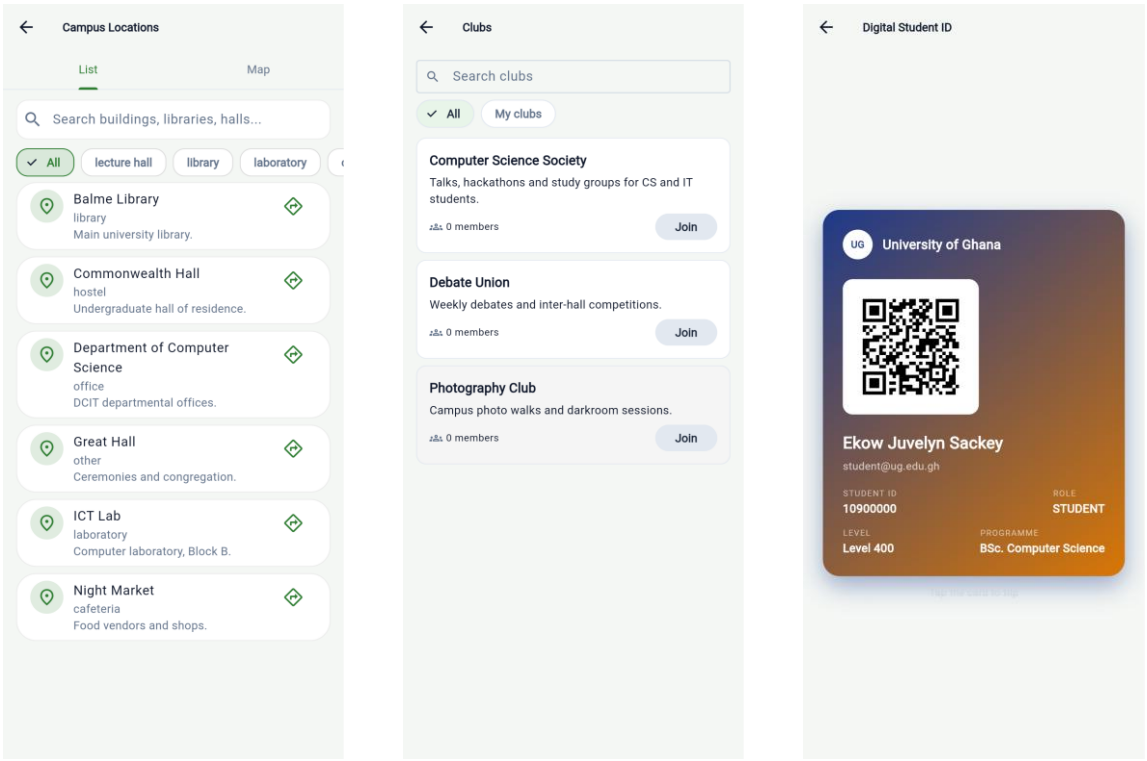
The campus map provides a custom map interface and a list of locations organised by category, and users can open supported locations in an external mapping application. The application

also estimates walking time between consecutive classes, but the estimate is based on straight-line distance between stored locations rather than live pedestrian routing.

The events module supports event creation, registration, cancellation and deletion, subject to the applicable permissions: event deletion is restricted to administrators or the lecturer who created the event. The clubs module supports club discovery, searching, viewing details, joining and leaving.

The announcements module provides an alerts feed and supports administrator broadcasts. Foreground polling is used to retrieve updates while the application is active. The digital identification module presents a flippable card with a QR code.

These modules demonstrate the breadth of the application's campus-life functionality. However, a feature's presence in the application should not be confused with complete operational integration. The Student Hub and Resources sections still rely on static or fixture-based content in important areas and are not connected to dedicated backend data structures.



(a) Campus locations (b) Clubs (c) Digital student ID

Figure 4.3: Campus-life features

4.4 Automated Testing Results

Automated testing was used to examine application logic, backend functions, database operations and API workflows (Section 3.5.3).

4.4.1 Summary of Test Results

The results recorded on 30 October 2026 are presented in Table 4.1.

Table 4.1: Summary of automated testing results

Test suite	Tests passed	Total reported	Outcome
Flutter automated tests	371	371	Passed
Backend offline unit tests	50	50	Passed
Backend database tests	14	14	Passed
Backend API smoke tests	182	182	Passed
Total test executions	617	617	All reported executions passed

The four suites account for 617 passing test executions. This is the sum of the suite totals and does not necessarily represent 617 unique test cases. The backend figures, and the first run of the mobile suite (366 tests), matched the independent snapshot taken on 24 October 2026 (Section 3.5.4); the mobile suite then grew to 371 tests when five tests were added (Section 4.4.2).

The results indicate that all reported test executions passed under their respective test environments. This provides evidence that the behaviours covered by those tests were functioning as expected at the time of the recorded runs. However, the results should be interpreted within the scope of the test suites. A passing test suite does not prove the absence of defects in untested scenarios, nor does it establish that every combination of user permissions, input values, network conditions and external services has been evaluated.

4.4.2 Flutter Automated Tests

The Flutter test suite recorded 371 passing tests out of 371 reported tests.

The tested areas include grading and prediction logic, calendar export, timetable parsing, assistant behaviour (including partial loss of data sources), campus-map calculations, notification-related logic, past-paper extraction, the sign-in and onboarding screens, and the dashboard shortcuts. Five tests were added during this evaluation: three regression tests for the defects described in Section 4.4.7, and two for the new dashboard shortcuts.

These tests help verify important parts of the mobile application's business logic. For example, testing grading calculations helps detect inconsistencies in the interpretation of score ranges,

while calendar export tests help verify the creation of supported calendar data. The result provides positive evidence for the mobile functions covered by the suite. It does not independently verify the complete appearance, accessibility, usability, or behaviour of every screen on every supported physical device.

4.4.3 Backend Unit Tests

The backend offline test suite recorded 50 passing tests out of 50 reported tests.

The suite covers grading-scale behaviour, push-notification fan-out and sender logic, and the verification of Google sign-in tokens, using test doubles in place of external services. Using test doubles allows selected backend functions to be examined without requiring every test to contact a real external service. The result indicates that the tested backend logic behaved as expected under the configured test conditions. It does not establish successful communication with every external service or guarantee that the same behaviour will occur under all production conditions.

4.4.4 Database Integration Tests

The database test suite recorded 14 passing tests out of 14 reported tests against a local PostgreSQL instance.

The tests cover database operations involving device tokens and push-notification preferences, and the linking of Google sign-in identities to accounts (creating a new account, linking an existing one by email, and rejecting an attempt to link one Google account to two users). These tests are important because unit tests that use simulated dependencies cannot, by themselves, confirm that SQL operations execute correctly against a real database. The result provides evidence that the specific database operations covered by the suite executed successfully in the local test environment. It does not constitute a complete assessment of database performance, recovery, security, or behaviour under production workloads.

4.4.5 API Smoke Tests

The backend API smoke-test suite recorded 182 passing tests out of 182 reported tests against the configured local database environment.

Smoke tests provide a broad check that selected API workflows can execute without major failures. The suite walks authentication, role permissions, events, clubs, announcements, device registration and the quiz lifecycle. It also includes one check against a live external service: a deliberately invalid Google sign-in token is sent to the real Firebase project and is

rejected as invalid rather than as unconfigured, which confirms that the credential itself is live and correctly wired.

The recorded result supports the conclusion that the tested API workflows were functioning in the local environment. It does not demonstrate that the application has been deployed successfully to a production server, that every endpoint has been exhaustively tested, or that the backend can support a particular number of simultaneous users.

4.4.6 Static Analysis

Flutter static analysis reported no issues in the recorded run. Static analysis helps identify certain classes of programming errors and code-quality problems without requiring every application path to be executed. The result is a useful indicator of the codebase's current static-analysis status, but it is not a substitute for runtime testing, security assessment, or user evaluation.

4.4.7 Defects Found by Running the Application

Running the complete application to capture the screenshots in this chapter exposed three defects that the passing automated suites had not caught. First, on a phone-width screen the dashboard's title was truncated to “Da…” because six unlabelled shortcut icons shared the top bar with it; the shortcuts were moved into a labelled grid on the dashboard (Figure 4.2(a)). Second, a rejected login, such as a mistyped password, replaced the whole screen with a full-page error and a Retry button, so the login form and its explanatory message disappeared: the application's root screen treated any session error as a failure to restore a saved session. Third, finishing or skipping the profile step returned the student to the institution picker instead of continuing to the login screen, because only the topmost onboarding screen was closed. All three were fixed, and the second and third are now covered by regression tests that were first confirmed to fail against the defective code.

The second and third defects are instructive. Each screen involved already had passing tests, because each was tested in isolation; both bugs only appeared when the screens ran inside the application's real root screen and navigation. For that reason, the new regression tests drive the application from its root rather than from the individual screen.

4.5 Evaluation Against Project Objectives

The specific objectives set out in Section 1.4.2 provide a basis for assessing the extent of implementation achieved (Table 4.2).

Table 4.2: Assessment of the specific objectives

Objective (Section 1.4.2)	Current evidence	Assessment
1. Cross-platform client and REST backend with role-based authentication, including Google Sign-In	Flutter application for Android and the web; Express API with JWT sessions, bcrypt password hashing and backend role checks covered by smoke tests; Google sign-in verification checked against the live Firebase project.	Implemented; Google Sign-In awaits a console configuration step.
2. Local-first timetable, examinations and alerts with .ics/CSV import and export and reminders	Timetable parsing and calendar-export tests; local reminders; alerts feed.	Implemented; cross-device timetable synchronisation is not provided.
3. Grade and GPA analytics on the University of Ghana scale, consistent between client and backend	One grading-scale definition checked by both the backend and the mobile tests; the score-55 inconsistency corrected.	Implemented; prediction accuracy remains unevaluated.
4. Campus-life features: events, clubs, locations, quizzes and digital ID	Smoke tests for event permissions and the quiz lifecycle; mobile tests for clubs and the campus map; Figures 4.2 and 4.3.	Major modules implemented; Student Hub and Resources remain static.
5. AI assistant with a cloud model, a local fallback, and resilience to unreachable data sources	Assistant tests, including the loss of individual data sources; Figure 4.2(c).	Implemented; response quality and educational effectiveness unevaluated; not yet run against a live Gemini key.
6. Push notifications through Firebase Cloud Messaging	Fan-out and sender tests with test doubles; device-token database and smoke tests.	Implemented and partially verified; delivery to a physical device not yet verified.
7. Layered automated testing, including mutation testing	617 passing test executions across four suites; clean static analysis; mutation checks; regression tests for the defects in Section 4.4.7.	Achieved for the recorded suites; no load, security or usability evaluation.

The assessment indicates that the principal software development objectives have been substantially addressed in the working prototype. However, implementation status is not the same as complete deployment readiness. Some objectives have been achieved at the level of functional implementation and automated testing, while others require additional integration, validation or real-world evaluation. This distinction is particularly important for predictive analytics, AI-assisted learning, push notifications, Google Sign-In, and services that are not yet connected to the backend.

4.6 Discussion of Findings

4.6.1 Integration of Academic and Campus Services

One of the main outcomes of the project is the integration of several student-oriented functions into a single mobile application. Timetable management, examination information, grade

analysis, quizzes, campus locations, events, clubs, announcements and digital identification are available within the same application environment. This design reduces the need to navigate separate interfaces when accessing the functions supported by the prototype.

Nevertheless, the application should not be described as a complete replacement for official university information systems. Some functions rely on device-local data, while other sections still contain fixture-based content. Full institutional integration would require reliable data sources, appropriate synchronisation, and confirmation that the application reflects authoritative university records.

4.6.2 Consistency of Academic Calculations

The correction of the grading inconsistency demonstrates the importance of using consistent business rules throughout a software system. When grades are displayed in the mobile application and also used by backend calculations, the same grading scale should produce consistent results. Before the correction, the server graded on a simplified scale, so a score of 55 was shown as a fail while the GPA calculation counted it as a D+. The grading scale is now defined once and checked by both the backend and the mobile test suites (Section 3.5.2).

The corresponding automated tests provide evidence that the tested grading behaviour is consistent under the defined cases. Additional validation should still compare a broader range of scores, grade boundaries, semester combinations and GPA scenarios against the official grading requirements applicable to the intended institution.

4.6.3 AI Assistance and Fallback Behaviour

The AI Assistant combines application context with an optional external generative AI service and a local keyword-based fallback. This design provides two modes of interaction and reduces dependence on the external service for every supported assistant operation. However, the fallback is limited and should not be treated as a full substitute for generative AI. Similarly, the availability of contextual information does not guarantee that every generated answer is correct or appropriate.

Future evaluation should measure response correctness, relevance, response time, failure handling, and the extent to which users find the assistant helpful. Any claims about improved learning outcomes would require a suitable user study rather than relying on implementation or unit-test results alone.

4.6.4 Reliability of Notification and Sign-in Integrations

The project includes notification-related functionality at both the application and backend levels. Automated tests cover selected notification logic, and database tests cover device-token and preference operations. The Firebase project is configured and its credential has been confirmed live through the sign-in check in Section 4.4.5. Nevertheless, the complete push-notification process has not yet been verified on a physical Android device.

The remaining verification should establish that a device can register its token, that the appropriate notification preferences are applied, that announcements reach the intended recipient, and that notification behaviour is correct when the application is not in the foreground. Consequently, notification functionality should be described as implemented and partially verified, rather than as fully confirmed end-to-end delivery. Google Sign-In is in a similar position: its code and backend verification are complete, but a real sign-in cannot take place until the provider is enabled in the Firebase console.

4.6.5 Local-First Operation and Synchronisation

Local-first handling supports selected functions without requiring every operation to depend on a backend request. This can be useful in environments where connectivity is intermittent. However, local availability and data synchronisation are separate properties. The current timetable implementation treats device-local data as the primary source and does not use the backend schedule route as a synchronised backup. Therefore, the project has not established that timetable changes automatically propagate between devices or remain consistent across multiple sessions and installations.

Future work should define a clear source of truth, synchronisation rules, conflict handling and recovery behaviour before complete multi-device timetable synchronisation can be claimed.

4.6.6 Technical Quality and Test Coverage

The recorded automated test results are encouraging because they cover mobile logic, backend functions, database operations and API workflows, and clean Flutter static analysis further supports the current state of the codebase. Coverage is uneven, however: events, clubs, quizzes, locations, and the account lookup used by registration and login have no dedicated backend unit tests and are exercised only by the end-to-end smoke suite, which gives a less precise failure location than a unit test would.

The tests also do not cover every aspect of system quality. The project has not established formal load-test results, measured response-time benchmarks, a representative usability study,

or a production security assessment. The defects in Section 4.4.7 show a further gap: screens that each passed their own tests still failed when combined. The results should therefore be interpreted as evidence of functional correctness for the tested scenarios rather than proof of comprehensive reliability, scalability, security or user satisfaction.

4.7 Outstanding Issues and Deployment Readiness

Although the application contains a substantial set of implemented features, several items remain important before it can be considered ready for production use (Table 4.3).

Table 4.3: Outstanding issues and required verification

Issue	Current state	Required action
Physical-device push notifications	Firebase configuration and backend logic are present, but delivery to a phone remains unverified.	Complete the Android device walkthrough and verify delivery in foreground and background conditions.
Google Sign-In	Implemented and tested; the Google provider is not yet enabled.	Enable the provider and register the app's fingerprint in the Firebase console, then test on a phone.
Apple sign-in and guest mode	Shown on the sign-in screens but not implemented.	Implement both before the project defence, or remove the buttons.
Production deployment	Local backend and database testing has been performed; production deployment is not established.	Configure the hosting environment, deployment secrets, database and migration procedures.
Repository setup	No Git remote is configured in the recorded project state.	Configure a repository remote and the deployment workflow.
Timetable synchronisation	The mobile application uses local-first timetable data.	Define and implement backend synchronisation if cross-device consistency is required.
Student Hub and Resources	Important sections rely on static or fixture-based content.	Implement backend routes, persistent data structures and validation.
Academic prediction validation	Prediction logic exists, but no predictive-performance results are available.	Evaluate predictions against suitable reference data and document the limitations.
AI response quality	Assistant logic and external AI integration exist, but response quality has not been evaluated.	Test representative questions, factual correctness, failure handling and usefulness.
Security and operational readiness	Core authentication and permission mechanisms are implemented.	Conduct a production security review and verify logging, backups, configuration and data protection.
User evaluation	No formal user-study results are available.	Conduct usability testing with appropriate participants and document the findings.

These issues do not negate the implementation work completed. Instead, they identify the difference between a technically substantial working prototype and a fully validated production application. The most immediate priorities are to complete end-to-end notification and sign-in

verification, establish the deployment environment, confirm production database and configuration procedures, and address the most important integration gaps. Academic prediction, AI quality and usability should then be evaluated using defined methods and measurable criteria.

4.8 Limitations of the Evaluation

The evaluation presented in this chapter is primarily based on software implementation and automated testing. It does not include measured academic outcomes, a controlled comparison with existing student applications, or results from a representative survey of university students.

No measured response-time or load-capacity figures are reported because such results have not been established. Similarly, the application has not been shown to improve GPA, reduce academic stress, increase productivity or improve examination performance.

The findings are also limited by incomplete end-to-end verification of push notifications and Google Sign-In, the local-first timetable design, partial backend integration of selected services, and screenshots taken from the web build rather than a physical device. Production deployment and operational behaviour remain to be verified. These limitations should be considered when interpreting the project's achievements; they also provide a clear direction for future work.

4.9 Chapter Summary

This chapter presented the results of implementing and testing Campus Companion. The application provides a range of academic and campus-life functions, including timetable and examination management, grade calculations, academic analytics, quizzes, AI assistance, campus locations, events, clubs, announcements and digital identification.

The results recorded on 30 October 2026 show 371 passing Flutter tests, 50 passing backend offline unit tests, 14 passing database tests and 182 passing API smoke tests, 617 passing test executions in total, and Flutter static analysis reported no issues. Running the complete application exposed three defects the suites had missed, which were fixed and are now covered by regression tests.

The findings indicate that the principal development objectives have been substantially addressed at the working-prototype level. However, the results do not establish complete

production readiness or demonstrate improved educational outcomes. Physical-device notification verification, Google Sign-In configuration, production deployment, timetable synchronisation, selected backend integrations, and formal evaluation of AI and prediction features remain outstanding. The next chapter presents the overall conclusions of the project, summarises its contributions, and recommends improvements for future development.

CHAPTER FIVE

CONCLUDING REMARKS

5.1 Summary of the Study

This study set out to address the fragmentation of academic and campus-life tools available to Ghanaian tertiary students (Section 1.2): students already rely heavily on general-purpose mobile tools (Chapter Two), but none of those tools is built around the specific conventions of their institutions, such as the University of Ghana's grading scale. In response, this study designed, built and evaluated Campus Companion, an integrated mobile application (Flutter) and backend service (Node.js/Express on PostgreSQL) spanning authentication and role-based access, a local-first timetable and examinations module, grade and GPA analytics on the correct local scale, campus events, clubs, locations and announcements, a quiz module, a digital student identity card, and an AI assistant designed to remain useful even when parts of its backend are unreachable (Chapter One, Section 1.3).

Its requirements were derived from the developer's own analysis of students' academic and campus-life needs (Section 3.1). The system was built iteratively, feature by feature, with the backend implemented and tested ahead of the corresponding mobile client code for each feature, and validated through a three-tier automated test strategy — offline unit tests, database-backed integration tests, and end-to-end smoke tests — reinforced by deliberate mutation testing of key business rules (Chapter Three). Re-running every tier specifically for this study (Chapter Four) produced 371/371 mobile tests, 50/50 backend unit tests, 14/14 database tests and 182/182 smoke-test checks. Measured against the seven specific objectives (Section 1.4.2), the principal development objectives were substantially met at the working-prototype level, while push delivery, Google Sign-In and the AI assistant's real-world behaviour remain to be verified (Table 4.2).

5.2 Contribution of the Study

Revisiting the three contributions claimed in Section 1.5 in light of Chapters Two to Four: first, the literature reviewed in Chapter Two did not surface an evaluated system that combines personal academic organisation and campus-wide information services behind one authenticated, institution-aware account (Section 2.8); Campus Companion's combination of these two strands, built around the University of Ghana's actual grading scale rather than a

generic one, fills that specific gap. Second, the AI assistant's design — reporting, source by source, which of its several independent backend data sources it could or could not reach, rather than failing an entire answer when one source is down — addresses a concern that is not the focus of the chatbot-in-education studies reviewed (Section 2.6), which evaluate whether chatbots help learning rather than how they behave when a service they depend on fails. Third, the test-driven development discipline demonstrated across this project — three test tiers, a shared cross-codebase fixture for a rule as consequential as GPA computation, and mutation testing used specifically to confirm that the test suite would catch a deliberately introduced regression — is documented here with concrete, repeatable results (Chapter Four) rather than asserted without evidence.

5.3 Limitations

Several limitations, most already flagged as they arose in earlier chapters, should be stated plainly together here. Two built and tested features — push notifications and Google Sign-In — are not yet observable in production: both depend on a small number of remaining steps outside the application's own code (enabling a sign-in provider and registering a fingerprint in the Firebase console; deploying a service-account key to the production server) rather than on unfinished application logic, but until those steps are completed neither delivers its real-world outcome to an end user (Section 4.7). The AI assistant's offline-resilience logic is verified against simulated failures but has not been exercised against a live large-language-model key or on a physical device (Section 4.6.3). Apple sign-in and a guest mode appear on the Welcome and Login screens but are not yet implemented; tapping either shows a short notice, and both are planned for completion before the project defence. The “Student Hub” shuttle, dues and lost-and-found features are simulated with static, device-local data and no real payment gateway (Section 1.3). The mobile client targets Android and web only; no iOS build exists. Several backend modules — events, clubs, quizzes, locations, and the find-or-create-by-email logic in registration and login — have no dedicated unit tests of their own and are exercised only by the end-to-end smoke suite, which is slower and less precise about failure location than a unit test would be (Section 4.6.6). Finally, a latent backend defect is documented but not triggered in current use: `POST /schedule` reads a `userId` from the request body that its own validation schema does not list, so the value is silently stripped and a class always lands on the caller's own timetable; this has no effect today because the mobile app is local-first and does not use the server timetable, but it would need fixing before that server-side timetable could be used as a cloud backup.

A formal usability evaluation involving representative university students was not conducted during this phase of the project. Although the application underwent functional and technical testing, these tests do not independently establish user satisfaction, ease of use or overall usability. Future work should include usability testing with students to gather feedback, identify usability challenges and guide further improvements to the application.

5.4 Recommendations for Future Work

1. Complete the remaining console and deployment steps for push notifications and Google Sign-In, then verify delivery and sign-in on a real device rather than only through the live-credential check used in this study (Section 4.7).
2. Exercise the AI assistant against a live large-language-model API key and on a physical device, to evaluate the quality of its answers rather than only the correctness of its fallback logic.
3. Replace the Student Hub's static fixtures with real backend tables and routes for shuttles, dues and lost-and-found, including a genuine payment integration if dues payment is to be supported.
4. Add dedicated backend unit tests for events, clubs, quizzes, locations and the registration/login find-or-create-by-email logic, to narrow the smoke suite's role to true end-to-end checks rather than the only safety net for these modules.
5. Implement Apple sign-in and the guest mode already shown on the Welcome and Login screens.
6. Conduct usability testing with university students to gather feedback, identify usability challenges and guide further improvements to the application.
7. Test user journeys through the application's root screen, not only screen by screen: two of the three defects found in Section 4.4.7 passed every screen-level test and appeared only when the screens ran inside the real application.
8. Fix `POST /schedule`'s handling of `userId` if the server-side timetable is ever to be used as a cloud backup for the (currently local-first) mobile timetable.
9. Consider an iOS build, since the current client and backend architecture does not depend on any Android-specific capability apart from the push and Google Sign-In integrations, which have iOS equivalents in Firebase.

5.5 Conclusion

Campus Companion demonstrates that the fragmentation described in Section 1.2 can be addressed with a single, institution-aware application built on an architecture (Chapter Three) that keeps a student's personal academic data local-first where appropriate, treats server-verified identity and role as the only trustworthy source of authorisation, and degrades gracefully rather than failing outright when one of several backend dependencies is unavailable. The system's correctness is demonstrated with repeatable, re-verifiable automated tests rather than asserted (Chapter Four), and its remaining gaps are configuration and scope decisions that are explicitly documented rather than hidden (Section 5.3). Future work (Section 5.4) is primarily about completing deployment of features that are already built and tested, rather than about redesigning the system described in this study.

REFERENCES

[APA 7th edition, alphabetical by first author. These 19 entries cover every source cited in this study. Each was independently verified (author names, year, venue) before being cited — none is invented. Confirm venue/DOI details against the original source before final submission, and consider managing this list in Mendeley or Zotero from here on.]

Adzifome, N. S., & Agyei, D. D. (2022). Learning with mobile devices: Insights from a university setting in Ghana. Education and Information Technologies. https://doi.org/10.1007/s10639-022-11300-4

Afful, D., & Boateng, J. K. (2023). Mobile learning behaviour of university students in Ghana. Cogent Social Sciences, 9(1), Article 2204712. https://doi.org/10.1080/23311886.2023.2204712

Baras, K., Soares, L., Vale Lucas, C., Oliveira, F., Paulo, N. P., & Barros, R. (2018). Supporting students' mental health and academic success through mobile app and IoT. International Journal of E-Health and Medical Communications, 9(1), 50–64. https://doi.org/10.4018/IJEHMC.2018010104

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for Agile software development. https://agilemanifesto.org/

Chen, E., Pei, Y., Chen, S., Tian, Y., Kotcher, R., & Tague, P. (2014). OAuth demystified for mobile application developers. In Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS '14) (pp. 892–903). ACM. https://doi.org/10.1145/2660267.2660323

Essel, H. B., Vlachopoulos, D., Tachie-Menson, A., Johnson, E. E., & Baah, P. K. (2022). The impact of a virtual teaching assistant (chatbot) on students' learning in Ghanaian higher education. International Journal of Educational Technology in Higher Education, 19, Article 57. https://doi.org/10.1186/s41239-022-00362-6

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Hardt, D. (Ed.). (2012). The OAuth 2.0 authorization framework (RFC 6749). Internet Engineering Task Force. https://doi.org/10.17487/RFC6749

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

MyStudyLife. (n.d.). MyStudyLife: The ultimate student productivity tool. Retrieved October 31, 2026, from https://mystudylife.com/

Okonkwo, C. W., & Ade-Ibijola, A. (2021). Chatbots applications in education: A systematic review. Computers and Education: Artificial Intelligence, 2, Article 100033. https://doi.org/10.1016/j.caeai.2021.100033

OWASP Foundation. (2021). OWASP Top 10: 2021. https://owasp.org/Top10/

Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track (pp. 81–91). USENIX Association.

Samarin, N., Sanchez, A., Chung, T., Juleemun, A. D. B., Gilsenan, C., Merrill, N., Reardon, J., & Egelman, S. (2024). The medium is the message: How secure messaging apps leak sensitive data to push notification services. Proceedings on Privacy Enhancing Technologies, 2024(4), 679–696. https://doi.org/10.56553/popets-2024-0151

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

University of Ghana. (n.d.). Sakai learning management system. Retrieved October 31, 2026, from https://sakai.ug.edu.gh/

Wang, Y., & Yan, X. (2018). Research and implementation of wisdom campus system based on Android platform. In Proceedings of the 2017 7th International Conference on Education and Management (ICEM 2017) (pp. 151–154). Atlantis Press. https://doi.org/10.2991/icem-17.2018.33

Wollny, S., Schneider, J., Di Mitri, D., Weidlich, J., Rittberger, M., & Drachsler, H. (2021). Are we there yet? A systematic literature review on chatbots in education. Frontiers in Artificial Intelligence, 4, Article 654924. https://doi.org/10.3389/frai.2021.654924

Zhao, Y., & Pardo, A. (2017). Evolving the design of a mobile application to support transition to tertiary education. Frontiers in ICT, 4, Article 25. https://doi.org/10.3389/fict.2017.00025